

ATOCO · IP2 大气取水机 · Opta 网关

opta-gateway-mt 软件架构说明

双线程网关固件：PLC 线程 + 云端主线程 + 喂狗线程

固件版本：a43c01d (批 0, 2026-09-04 经 OTA 上板)

仓库：ATOCO_CONTROL/opta-gateway-mt/ (独立 git)

硬件：Arduino Opta (STM32H747 Cortex-M7, mbed OS 6, Arduino mbed core 4.6.0)

读者：需要看懂整个数据流、线程边界和故障行为的工程师

生成日期：2026-09-04

本文按"先整体、再逐层、最后故障与演进"的顺序组织。每一节都能独立阅读；图中颜色固定：主线程 PLC 线程 喂狗线程 共享面。

目录

1. 为什么要拆成两个线程
2. 系统全貌
3. 线程与所有权
4. 文件地图
5. 共享面 shared.h: 两个线程唯一的接触点
6. PLC 线程: 每 2 秒一拍
7. 主线程: setup() 与 loop()
8. 快照的一生: 从 PLC 标签到云端属性
9. 喂狗线程: 看两条心跳, 怪较旧的那条
10. 两个探针: 先敲门, 再进门
11. 启动序列
12. 四种故障与固件的反应
13. 云端属性清单
14. 与旧固件的对比、批次路线图
15. 批 1 之前要知道的事
16. 术语表

一句话版本。PLC 线程独占 EtherNet/IP 客户端, 每 2 秒把 28 个传感器读进一个约 220 字节的纯数据快照, 加锁拷贝一次发布出去; 主线程独占全部云端属性和 `ArduinoCloud.update()`, 每趟 loop 加锁把快照拷回来, 序号变了就逐槽赋给云属性。两个线程各打各的心跳, 喂狗线程取较旧的那条判断有没有卡死。任何一边阻塞, 另一边照常工作。

1. 为什么要拆成两个线程

旧固件（`opta-plc-gateway-ip2.ino`，2587 行单文件）是单线程：`loop()` 里依次做 PLC 读取、云端 `update()`、日志、诊断。ArduinoIoTCloud 库在 `update()` 内部默认武装一个 **32.76 秒** 的硬件看门狗，并且**只在 `update()` 里喂**。于是 `loop()` 里任何一处超过 32 秒的阻塞都会让板子被硬件复位。

实测两处这样的阻塞：

阻塞点	触发条件	实测时长	后果
<code>eip.begin()</code> （PLC 的 TCP 连接）	PLC 断电或网线拔掉	27.7 s	逼近看门狗窗口；与其他延迟叠加即复位
ArduinoCloud.update() 内的 TLS 重连	WiFi 通、但到 <code>iot.arduino.cc</code> 的链路断	110 s 以上	直接复位；过夜 6 次重启全部落在这里

先后加过三层缓解：独立喂狗线程、PLC 探针（先用 300 ms 的 TCP 敲门再 `begin()`）、云端探针（先敲 8885 端口再进 `update()`）。重启从 13–77 分钟一次降到 2–4 小时一次，但没归零，因为根子在于**一个线程里放了两个各自都可能长阻塞的 I/O 对象**。

结构性解法。把两个会阻塞的对象分到两个线程：Ethernet/IP 客户端归 PLC 线程，云端连接归主线程。一边阻塞只卡自己，喂狗线程仍能看到另一边活着。这就是 `opta-gateway-mt`。

设计原则（贯穿全部代码）

- **每个可阻塞对象只有一个 owner。**`eip` 只在 PLC 线程被调用；`Cloud*` 属性只在主线程被赋值。调试构建里有断言宏在违规时打印并自旋。
- **跨线程只传纯数据。**快照里没有 `String`、没有指针，用 `static_assert(is_trivially_copyable)` 在编译期锁死。原因：一个线程赋值 `String` 的同时另一个线程在序列化它，就是 `use-after-free`。
- **锁里只做一次结构体拷贝。**不做 I/O、不做分配。全仓库只有一把锁、四个持锁函数、每个的临界区就是一行拷贝。
- **主线程永远不等 PLC 线程。**命令队列用非阻塞的 `try_alloc / try_get`，队列满了就丢并计数。
- **先敲门再进门。**任何无界的 `connect` 之前，先用带超时的原生 TCP socket 探一下端口开不开。

2. 系统全貌

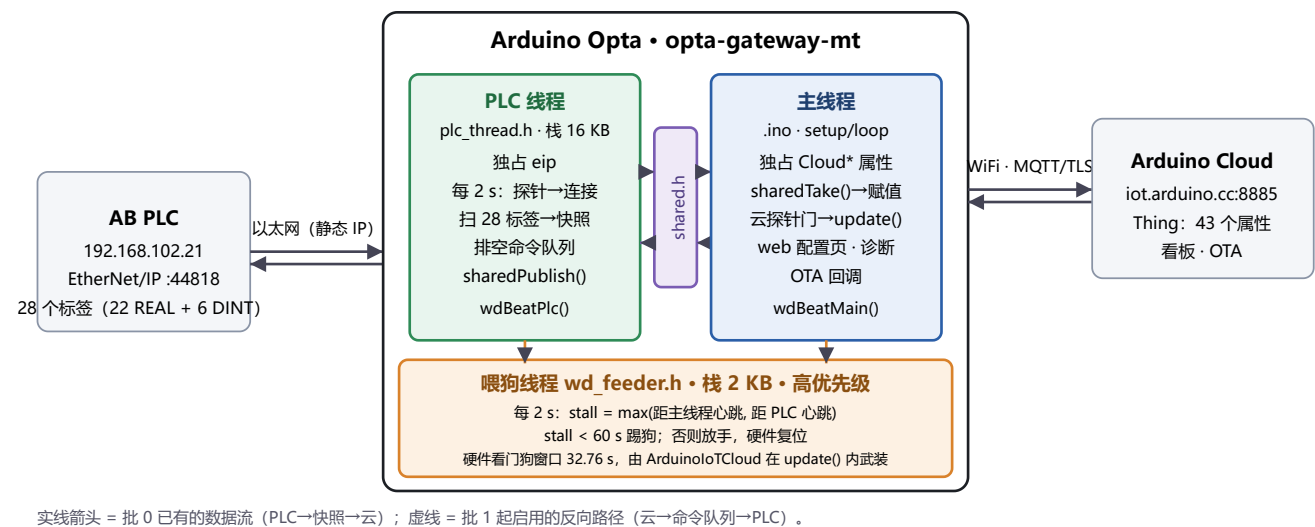


图 2-1 系统全貌。以太网是 PLC 链路，WiFi 是云链路；两条链路各归一个线程。

板上跑着三个我们自己的线程，外加 mbed/Arduino 核心自己的线程（网络栈、USB 等）。板和 PLC 之间走以太网口静态 IP，不设网关；默认路由在 WiFi 上，所以云流量走 WiFi。批 0 是只读骨架：数据只从 PLC 流向云，云到 PLC 的命令路径（虚线）已经建好队列但还没有人往里写。

3. 线程与所有权

线程	优先级 / 栈	周期	独占的对象	绝不碰的东西
主线程 setup()/loop()	Normal / 默认	loop() 自由运转，通常每趟几毫秒；诊断块每 30 s，串口心跳每 3 s	全部 43 个 Cloud* 属性；ArduinoCloud；WiFi；web 配置服务器（以太网口 80 端口）；串口配置；面板 LED；主线程栈水位	eip 的任何方法
PLC 线程 plcThreadBody()	Normal / 16384 B	固定 2000 ms 一拍（SAMPLE_INTERVAL_MS），超时的一拍结束后立刻开始下一拍	eip（EtherNet/IP 客户端及其 5 个 700 B 静态缓冲）；PLC 探针；本地工作快照 w；命令队列的消费端	任何 Cloud*、ArduinoCloud、WiFi
喂狗线程 wdFeederLoop()	High / 2048 B	每 2000 ms 醒一次	硬件看门狗的 kick()；stall 统计	一切 I/O。它只读两个 32 位时间戳

为什么 PLC 线程要 16 KB 栈

旧固件里 pollSensors() 跑在主线程的默认栈上，加上 CIP 报文构造和 TLS，栈水位曾逼近告警。拆出来之后 PLC 线程只跑 CIP，16 KB 是保守取值；EtherNet/IP 客户端的收发缓冲是函数级 static，放在 .bss 不占栈；快照 w 约 220 字节在线程栈上。实际水位由 StackWatch 测量并放进快照的 plcStackFree 字段（批 0 只在串口显示）。

调试期的归属断言

```
SHARED_ASSERT_ON_PLC(); // pollSensorsInto() 第一行：不在 PLC 线程上就打印并自旋
SHARED_ASSERT_ON_MAIN(); // cloudSideAssign() 第一行：不在主线程上就打印并自旋
```

g_plcThreadId 在线程体第一行记录，g_mainThreadId 在 setup() 里记录。ENABLE_SERIAL_DEBUG=0 时两个宏为空。违规后自旋，喂狗线程 60 秒后放手，硬件复位——在台架上是最响的失败，在现场也不会永久挂死。

4. 文件地图

文件	行	归属	职责
opta-gateway-mt.ino	541	主	主线程文件。定义 eip（但不调用）、WiFi/KVStore 配置、web 配置页、WiFi 救援、OTA 回调、面板 LED、setup()、loop()。
plc_thread.h	163	PLC	PLC 线程本体：重连门（探针 + 指数退避）、pollSensorsInto()、drainCommands()、发布、plcThreadBegin()。
cloud_side.h	62	主	唯一从 PLC 数据写 Cloud* 的地方：cloudSideConsume() 取快照，28 个 TAKE(槽，属性) 赋值，lastError 在此构造 String。
shared.h	175	共享	两个线程唯一的接触面：PlcSnapshot、CtrlState、Cmd 队列、一把互斥锁、六个访问函数、归属断言。
wd_feeder.h	97	喂狗	双心跳喂狗线程、WD_AT_* 位置码 0-13、stall 最大值与归因。
stack_watch.h	55	各线程	手工栈水位：在线程最浅点把栈下方刷成 0x5A5A5A5A，之后数还剩多少没被覆盖。struct，每线程一个实例。
plc_tags.h	160	只读表	28 个 PLC 标签名、6 个 DINT 的缩放/偏移、REAL/DINT 到规范槽位的映射，6 条 static_assert。
thingProperties.h	115	主	43 个云属性的声明和 addProperty 注册（权限、策略、阈值）。名字与旧固件一致，用同一个 Thing。
plc_probe.h	82	PLC	300 ms 的 TCP 敲门：44818 端口有人听才让 eip.begin() 去连。
cloud_probe.h	126	主	500 ms 的 TCP 敲门 iot.arduino.cc:8885，加 5 s 节流；cloudUpdateAllowed() 决定这趟要不要进 update()。
EtherNetIP.h / .cpp	124 / 526	PLC	自研 EtherNet/IP 显式报文客户端：会话注册、CIP 读写、12 标签一组的 MSP 批读，单次 I/O 超时 1500 ms。旧固件逐字拷贝。
boot_reason.h	313	主	启动原因：读 RCC->RSR、KVStore 里的启动计数 n=。云端 bootReason 字符串就来自这里。
serial_config.h	136	主	串口改 WiFi 凭据的小协议（存进 KVStore）。
config.h	1109	配置	全部编译期常数。批 0 把 8 个功能开关强制为 0（阀、PLC 状态、水量、AI、热泵、自动吸附、USB 日志等）。
tools/build.sh	—	工具	唯一受支持的构建入口：从 git 生成 version.h（FW_VERSION），再调 arduino-cli。

读代码的建议顺序。 shared.h（合同）→ plc_thread.h（生产者）→ cloud_side.h（消费者）→ .ino 的 setup()/loop()（编排）→ wd_feeder.h（保险）。config.h 和两个探针按需查。

5. 共享面 shared.h：两个线程唯一的接触点

所有跨线程的东西都在这一个文件里：一个 PLC→主的快照、一个主→PLC 的连续控制块、一个主→PLC 的一次性命令队列、一把锁。设计文档的判据是：**以后的改动如果能不碰这个文件，说明线程边界画对了。**

5.1 PlcSnapshot (PLC → 主，每拍一份)

字段	类型	含义
seq	uint32	单调递增，每次发布 +1， 在锁内 递增，所以主线程不可能看到新序号配旧数据。
stampMs	uint32	发布时的 <code>millis()</code> 。主线程用它算快照年龄，超过 3 拍 (6 s) 判定 PLC 线程卡住。
sensor[28]	float	28 个传感器值，槽序 = <code>SENSOR_TAGS</code> 顺序 (见第 13 节)。
ok[28]	bool	这一拍哪些槽读到了。主线程只赋值 <code>ok</code> 为真的槽，读失败的属性保留上一个真值而不是变成 0。
fails	int32	本拍没读到的槽数。
plcConnected	bool	EtherNet/IP 会话是否在。
eipMs	int32	上一次 <code>eip.begin()</code> 尝试耗时 (含探针)。
lastCipStatus	uint16	最近一次 CIP 通用状态码。
lastError[48]	char	"ok" / "read fail x3 (cip=0x5)" / "plc not listening (probe)"。是 <code>char[]</code> 不是 String 。
plcStackFree	uint32	PLC 线程栈的最小剩余 (字节)。
cmdDropped	uint32	因队列满而丢弃的命令累计数。

整个结构约 220 字节，`static_assert(sizeof < 1024);` `static_assert(is_trivially_copyable)` 保证没有 String/指针混进来。

5.2 CtrlState (主 → PLC，连续) 与 Cmd 队列 (主 → PLC，一次性)

<pre>struct CtrlState { bool controlEnabled; // 批 1 float flowRate; // 批 2 float flowBatch; // 批 2 };</pre>	<pre>struct Cmd { uint16_t tag; // CmdTag 枚举，24 项 bool isBool; float value; }; rtos::Mail<Cmd,16> g_cmdQueue;</pre>
---	---

批 0 没有人写它们，但结构第一天就定型，后面批次不必再改这个文件。命令队列是 mbed 的 `Mail`，16 槽；投递用 `try_alloc`，满了返回 `false` 并把 `g_cmdDropped +1`；取用 `try_get`，空了立刻返回。**两端都不阻塞。**

5.3 锁的规则

<pre>sharedPublish(w): stampMs=millis(); lock(); w.seq=published.seq+1; published=w; unlock(); sharedTake(out,&lastSeq): lock(); out=published; unlock(); 若 out.seq==lastSeq 返回 false</pre>

一把 `rtos::Mutex` (带优先级继承) 守两个结构体。四个持锁函数的临界区都只有一行结构体拷贝，微秒级。两把锁只会多出一种死锁方式，所以只用一把。

6. PLC 线程：每 2 秒一拍

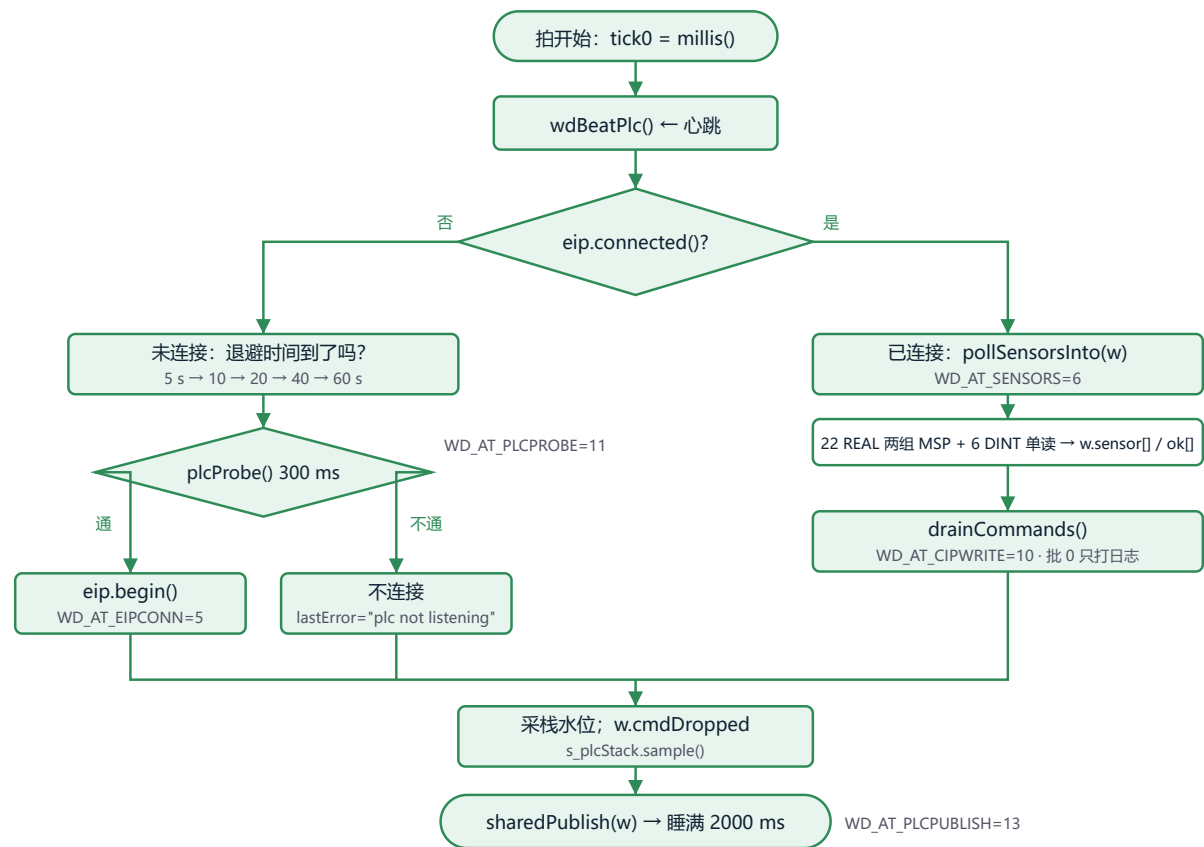


图 6-1 PLC 线程的一拍。左分支是未连接时的重连门，右分支是已连接时的扫描。两条分支都以发布收尾，所以**即使 PLC 掉线，快照也每 2 秒更新一次**（带 `plcConnected=false`）。

扫描细节 `pollSensorsInto(w)`

- 先把 28 个 `ok[]` 清零。
- 22 个 REAL 用 Multiple Service Packet 批读：`EIP_MAX_MSP_TAGS=12`，分两组两次往返。结果按 `REAL_SLOT[k]` 散射进规范槽位。
- 6 个 DINT（4 个热泵压力原始计数、2 个 DS 温湿度）单个读，`raw × DINT_SCALE + DINT_OFFSET` 后散射进各自槽位。
- 统计 `fails`；失败集合变化时才在串口列出失败标签名（不是每拍都刷屏）。
- 全部 28 个都失败 → `eip.end()`，置 `plcConnected=false`，下一拍进入重连门。

重连门

退避从 5 s 起，每次失败翻倍，上限 60 s，成功后复位。**先探针再 begin**：探针是原生 `TCPSocket`，`set_timeout(300)` 在 `connect()` 之前调用，`netsocket` 层对 `connect` 生效，所以 PLC 断电时一次尝试只花 0.3 s，而不是旧固件的 27.7 s。

7. 主线程：setup() 与 loop()

7.1 setup() 的顺序（每一步的位置都有理由）

#	调用	为什么在这里
1	<code>bootReasonCapture()</code>	必须第一句：读 RCC->RSR，越晚越可能被别的初始化清掉。
2	<code>Serial.begin(115200)</code> ，最多等 1.5 s	有上限，不会因为没插 USB 而卡死。
3	<code>g_mainThreadId = osThreadGetId()</code>	让 <code>SHARED_ASSERT_ON_MAIN</code> 有参照。
4	以太网静态 IP + <code>cfgServer.begin()</code>	PLC 链路和 web 配置页都在以太网口；静态 IP、无网关，默认路由留给 WiFi。
5	<code>loadWifiConfig()</code> → <code>WiFiConnectionHandler</code> （超时表 5 s）→ <code>initProperties()</code> → <code>fwVersion/bootReason</code> 赋值 → <code>ArduinoCloud.begin()</code> → OTA 回调	云侧全部在主线程初始化； <code>begin()</code> 不阻塞，真正的连接在 <code>update()</code> 里进行。
6	三个面板 LED <code>pinMode</code>	—
7	<code>plcThreadBegin()</code>	启动 PLC 线程。 setup() 里不做 <code>eip.begin()</code> ：第一次连接由 PLC 线程在自己的栈上、经探针完成。
8	<code>wdFeederBegin()</code>	在 PLC 线程之后启动；喂狗循环开头把两个心跳都置为当前时间，不会误判。
9	<code>s_mainStack.begin()</code>	最后一句：在最浅栈点刷水位图案。

7.2 loop() 的一趟

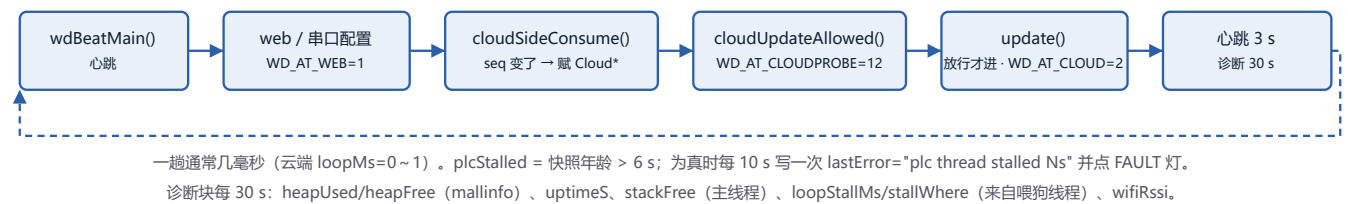


图 7-1 主线程 `loop()` 一趟。没有任何一步等待 PLC 线程。

`cloudSideConsume()` 每趟都调，但只有 `seq` 变化时才做 28 次赋值——所以云属性每 2 秒更新一次，与 PLC 拍同步，而不是每趟 `loop` 都刷。

8. 快照的一生：从 PLC 标签到云端属性

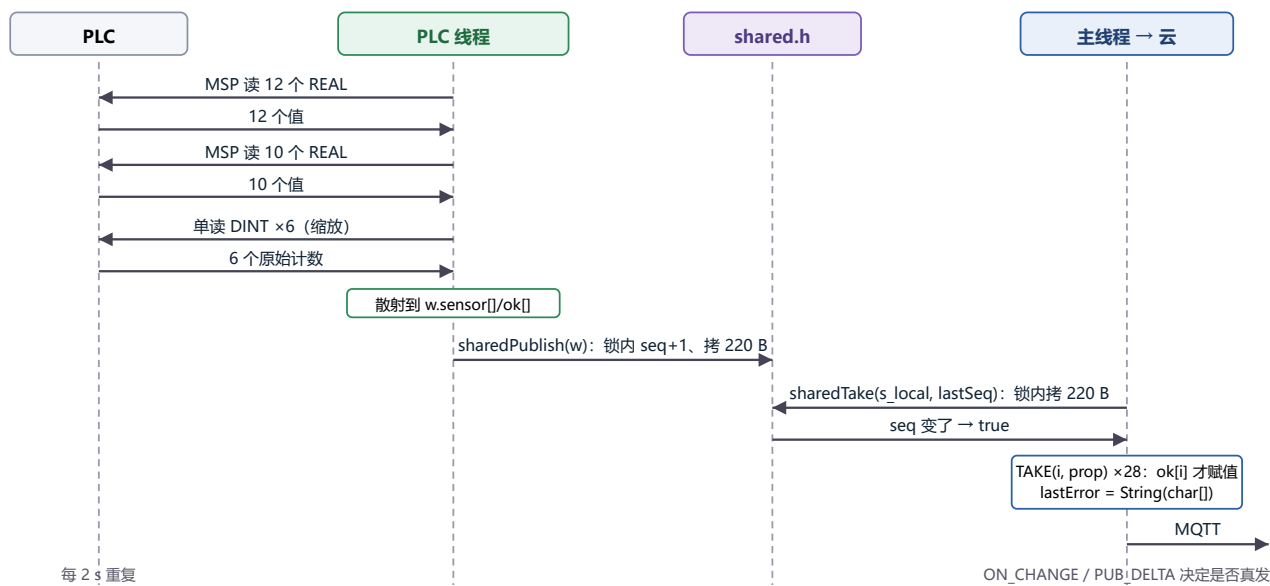


图 8-1 一拍数据的旅程。两次加锁各只拷一个结构体；String 只在主线程、在赋值瞬间构造。

槽位是合同

plc_tags.h 的 SENSOR_TAGS[] 顺序定义了槽 0–27。PLC 线程按这个顺序写 w.sensor[]，cloud_side.h 的 28 个 TAKE(i, 属性) 按同一顺序读。两边都靠这一张表；表的长度有 static_assert(N_SENSORS == 28) 把守，一对一配对的正确性在代码审查时逐槽核过（见第 13 节）。

"读失败不归零"

旧固件读失败时属性会被写成 0，看板上出现假的归零尖峰。现在 TAKE 只在 ok[i] 为真时赋值，失败的槽保持云端上一个真值，同时 fails、lastError 说明发生了什么。

云端只在变化时发

28 个传感器都注册为 READ, ON_CHANGE；压力、热泵压力和液位另加 PUB_DELTA 阈值抑制抖动。所以每 2 秒赋值 28 次并不等于每 2 秒发 28 条 MQTT，库只发变了的。

9. 喂狗线程：看两条心跳，怪较旧的那条

```
每 2000 ms:
sinceMain = now - s_mainBeatMs      // 主线程每趟 loop 开头打一次
sincePlc   = now - s_plcBeatMs      // PLC 线程每拍开头打一次
since      = max(sinceMain, sincePlc)
若 since 创新高: s_stallMaxMs = since; s_stallWhere = (sinceMain ≥ sincePlc) ? s_whereMain : s_wherePlc
若硬件看门狗还没被武装: continue    // 它由 update() 内的云状态机武装, setup 末尾还没跑
若 since < 60000: kick()             // 否则放手, s_wdRefusals++, 硬件在 32.76 s 内复位
```

为什么取 max: 只要有一个线程卡死，系统就已经不健康，喂狗线程不能替它遮掩。**为什么 60 s 放手:** 一个永远不回来的线程比一次重启更糟；60 s 给合法的长操作（PLC 半死时一拍最坏约 24–28 s，主线程 DNS 全灭时一趟最坏约 30 s）留了约两倍余量。

位置码 WD_AT_*

每个线程在进入一段可能阻塞的代码前登记自己在哪。两个线程各有一个槽（s_whereMain / s_wherePlc），归因时取较旧心跳所属线程的槽。如果只用一个共享槽，健康线程会在每趟结束时把它覆盖成 NONE，卡死现场就丢了——这正是审查阶段改掉的一个缺陷。

码	名字	线程	正在做什么
0	NONE	—	不在任何登记段
1	WEB	主	handleConfigClient() web 配置页
2	CLOUD	主	ArduinoCloud.update()。开机首次 TLS 连接约 6 s 就落在这里，所以云端 loopStallMs≈6000@2 是正常的
3	USB	—	USB 日志（批 6，当前编译掉）
4	PUSH	主	Influx 自检（批 3）
5	EIPCONN	PLC	eip.begin() 建立会话
6	SENSORS	PLC	pollSensorsInto() 扫 28 标签
7 / 8 / 9	VALVES / PLCSTATE / HEATPUMP	PLC	批 1 / 批 1 / 批 5
10	CIPWRITE	PLC	排空命令队列、写 PLC
11	PLCPROBE	PLC	300 ms 探针
12	CLOUDPROBE	主	500 ms 云探针（含 DNS）
13	PLCPUBLISH	PLC	sharedPublish()

码本身就说明了线程：1、2、4、12 只出现在主线程，5–11、13 只出现在 PLC 线程。云端属性 stallWhere 发布的就是这个码。

已知局限。 loopStallMs 只增不减：开机那次 6 s 的云连接会把它钉在 ~6000，之后一次 20 s 的停顿如果没超过历史最大值就看不见。批 1 计划改成滑动窗口。

10. 两个探针：先敲门，再进门

两处无界阻塞的共同病因是：Arduino 的 `Client` 在 `connect()` 期间是阻塞的，只在连接成功后才被设成非阻塞；对方不在时就跑满协议栈的 TCP 重传（约 28 s）。两个探针用同一个药方：用原生 `MBED` `TCP Socket`，在 `connect()` 之前 `set_timeout()`，`netsocket` 层对 `connect` 生效，然后只开一下、关掉，什么都不发。

PLC 探针 `plc_probe.h`

目标 192.168.102.21:44818，超时 **300 ms**。
何时：PLC 线程重连门内、`eip.begin()` 之前。
不通 → 不连、写 `lastError="plc not listening (probe)"`、退避翻倍。
代价：PLC 断电时一次尝试 0.3 s，而不是 27.7 s。

云探针 `cloud_probe.h`

目标 `iot.arduino.cc:8885`（MQTT/TLS 端口），超时 **500 ms**，DNS 结果缓存。
何时：主线程每趟 `loop`，只在“WiFi 已连、云未连”这一个状态才真敲；5 s 节流。
不通 → 这趟跳过 `update()`（反正只会卡住）。

云探针的门为什么这么窄

```
cloudUpdateAllowed(wifiUp, cloudUp, now):
  cloudUp  → true    // 已连: update() 负责发布、keepalive、OTA，绝不能跳
  !wifiUp  → true    // 无线断: update() 是唯一会重新关联射频的东西，跳了就永远离线
  否则 (WiFi 通、云断)：每 5 s 敲一次 8885；敲通才 true
```

只有“WiFi 通、云断”这一个状态的下一轮 `update()` 是那个无界的 TLS `connect`；其他状态的 `update()` 都很快或者必须跑。DNS 失败按“不通”处理：DNS 死通常意味着互联网断了，这时进 `update()` 只会白卡。

实测归因（改造前的过夜数据）

六次重启里有两次 WiFi 信号稳定在 -70 dBm 长达 8 分钟，说明掉的不是无线链路而是到 `iot.arduino.cc` 的上游。所以探针测的是“broker 端口能不能到”，而不是 RSSI 这个有三分之一时间指错方向的代理。

已知局限。 DNS 失败时 `s_brokerResolved` 保持 `false`，每次敲门都重跑 `gethostbyname`（`MBED` 默认 5 s × 3 次），互联网断开期间主线程每 5 s 可能阻塞 15–30 s——仍在 60 s 放手线之内，但批 1 应缓存否定结果。

11. 启动序列

接上串口（115200）时应按这个顺序看到下面的行；每一行对应 setup() 的一步或一个线程的起点。

[BOOT] rsr=0x... n=36 ...boot_reason.h: 复位原因 + KVStore 启动计数

[MT] opta-gateway-mt starting fw=a43c01d版本来自 build.sh 生成的 version.h

[ETH] instance=1 configured=192.168.102.107 actual=192.168.102.107

[CFG] web config page listening on port 80

[PLC] thread launched, stack 16384 Bsetup() 第 7 步

[WD] feeder started, kick every 2000 ms, give up after 60000 ms, watching main+plc

[PLC] thread startedPLC 线程体第一行

[EIP] session OK探针通 → begin() 成功 (PLC 关机时是 "plc not listening (probe)")

[HB] wifi=up cloud=up plc=1 seqAge=1830 cloudMs=1 eipMs=3 stall=5999@2 mainStk=26256 plcStk=... t1=44.8 lvl=20.2

[HB] 字段	含义与健康值
seqAge	当前快照年龄 (ms)。PLC 线程每 2 s 发布一次，所以健康时 < 2500， 不论 PLC 是否连着 。
cloudMs	这趟 update() 花的时间。稳态 0–1 ms。
eipMs	最近一次连接尝试耗时。PLC 在线约 3 ms；PLC 断电约 300 ms（探针拒绝）。
stall@where	历史最大 stall 与位置码。开机 ~6000@2 正常。
mainStk / plcStk	两个线程栈的最小剩余。都应 > 4000。

云端的对应表现（2026-09-04 12:00 首次上板实测）

属性	值	说明
fwVersion	a43c01d	与 git 提交一致，无 -dirty 后缀
bootReason	none rsr=0x00000000 n=36	OTA 重启本身让 n 从 35 变 36；此后不动即健康
plcConnected / eipMs / lastError	True / 3 / ok	PLC 会话 3 ms 建立
t1HotTank 等 28 项	2–7 s 前更新	PLC 线程每 2 s 一拍
loopStallMs / stallWhere	5999 / 2	开机 TLS 连接
stackFree	26256	主线程栈最小剩余

12. 四种故障与固件的反应

故障	PLC 线程	主线程 / 云端	看板与串口上的样子
PLC 断电或网线拔掉	下一拍 28 个全失败 → <code>eip.end()</code> ；之后每拍走重连门：探针 0.3 s 拒绝，退避 5→10→20→40→60 s。 每拍仍发布 (<code>plcConnected=false</code>)。	完全不受影响： <code>update()</code> 照常，诊断照常。 <code>cloudSideConsume</code> 每 2 s 仍收到新 seq，传感器属性因 <code>ok[]</code> 全假而保持旧值。	<code>plcConnected=False</code> , <code>lastError="plc not listening (probe)"</code> , <code>eipMs≈300</code> , <code>uptimeS</code> 持续更新；串口 <code>seqAge<2500</code> 、 <code>cloud=up</code> 。PLC 回来后 60 s 内 "session OK"。
WiFi 或上游断	完全不受影响：继续每 2 s 扫描发布。	WiFi 断：门放行， <code>update()</code> 快速返回并重新关联。WiFi 通、云断：每 5 s 敲一次 8885，不通就跳过 <code>update()</code> ，主线程不会卡进 TLS connect。	看板全部属性停更（板子发不出去）；串口 <code>wifi=down</code> 或 <code>cloud=down</code> 但 <code>plc=1</code> 、 <code>t1/lvl</code> 继续变化、 <code>stall</code> 不爬升。恢复后云端出现新值。
PLC 半死（端口开着、不回 CIP）	探针通 → <code>begin()</code> 或读超时：每次 I/O 1500 ms 超时，一拍最坏约 24–28 s（两组 MSP + 6 单读），仍在 60 s 内。	不受影响。快照年龄可能超过 6 s → <code>plcStalled</code> 为真， <code>lastError="plc thread stalled Ns"</code> ，FAULT 灯亮。	<code>lastError</code> 报 <code>stalled</code> ； <code>stall@6</code> 或 <code>@5</code> 增大。这是终审点名 最值得用串口验证 的场景： <code>readExact</code> 每收一字节就重置计时，慢速滴答的对端理论上可拖长一拍。
某线程真的卡死（bug、死锁）	喂狗线程看到该线程心跳超过 60 s → 停止踢狗 → 硬件在 32.76 s 内复位。 <code>s_stallWhere</code> 记下卡死线程当时的位置码。		重启后 <code>bootReason</code> 的 <code>n</code> 加一，云端 <code>loopStallMs/stallWhere</code> 在复位前的最后一次诊断里指向卡死段（如果那次诊断赶上发出去）。

面板 LED（批 0 语义）

LED	亮的条件
LED_D1 (PLC)	<code>plcConnected</code> 为真
LED_D2 (云)	<code>ArduinoCloud.connected()</code>
LED_D3 (故障)	PLC 未连 或 快照年龄 > 6 s。阀/温压故障灯要到批 1 才回来。

注意：`panelLeds()` 目前在串口心跳块里（`ENABLE_SERIAL_DEBUG` 为 1 时才刷），批 1 会把它移出来。

13. 云端属性清单

13.1 传感器 28 项（槽序 = PLC 线程写、主线程读的合同）

槽	PLC 标签	云属性	类型	读法	发布策略
0–11	Temp_1 ... Temp_12	t1HotTank ... t12HeatPumpIn	CloudTemperatureSensor	REAL MSP	ON_CHANGE
12	Vaisala_Conden_Temp	vtVaisalaTemp	CloudTemperatureSensor	REAL MSP	ON_CHANGE
13–17	Press_1 ... Press_5	p1TopChamber, p2BotChamber, p3ChamberRedun, p4Booster, p5Collector	CloudFloat	REAL MSP	ON_CHANGE + PUB_DELTA
18	Vaisala_Conden_WVP	p6VaisalaWvp	CloudFloat	REAL MSP	ON_CHANGE + PUB_DELTA
19–22	Raw_Module_1/2_Pressure_Grey/White	hp1HotInlet, hp1ColdInlet, hp2HotInlet, hp2ColdInlet	CloudFloat	DINT 单读	ON_CHANGE + PUB_DELTA
23	Atmosphere_Temp	atmTemp	CloudTemperatureSensor	REAL MSP	ON_CHANGE
24	Atmosphere_Humidity	atmRh	CloudRelativeHumidity	REAL MSP	ON_CHANGE
25	Atmosphere_Temp_02	atmTempDs	CloudTemperatureSensor	DINT 单读	ON_CHANGE
26	Atmosphere_Humidity_02	atmRhDs	CloudRelativeHumidity	DINT 单读	ON_CHANGE
27	Collection_Tank_Level	tankLevel	CloudFloat	REAL MSP	ON_CHANGE + PUB_DELTA

13.2 诊断 15 项（全部 READ）

属性	来源	策略	含义
plcConnected	快照	ON_CHANGE	EtherNet/IP 会话在不在
eipMs	快照	ON_CHANGE	最近一次连接尝试耗时
lastError	快照 → String	ON_CHANGE	"ok" / 读失败 / 探针拒绝 / PLC 线程 stalled
loopMs	主线程	每 5 s	一趟 loop 的时长
cloudMs	主线程	ON_CHANGE	一次 update() 的时长
fwVersion / bootReason	setup()	ON_CHANGE	固件哈希; 复位原因 + 启动计数 n=

stackFree	主线程 StackWatch	ON_CHANGE	主线程栈最小剩余
loopStallMs / stallWhere	喂狗线程	ON_CHANGE	历史最大 stall 与位置码
wifiRssi / heapUsed / heapFree / uptimeS	主线程诊断块	每 30 s	信号; mallinfo 堆; 开机秒数
otaPending	OTA 回调	ON_CHANGE	批 0 已注册但从不置真

属性名与旧固件完全一致，所以看板和 InfluxDB 转发无需改动；旧固件另外 112 个属性在批 0 期间停更并保留最后的值（见第 14 节）。

14. 与旧固件的对比、批次路线图

方面	旧固件 opta-plc-gateway-ip2	opta-gateway-mt 批 0
结构	单文件 2587 行 + 若干头文件，单线程 loop()	按职责拆成 12 个文件，三个线程，一个共享面
阻塞的传播	PLC 或云任一阻塞都会拖住另一个并触发看门狗	各自阻塞只卡自己；喂狗线程分别看两条心跳
读失败的表现	属性写成 0	保留上一个真值，fails/lastError 说明原因
云属性	155 个（含 16 个可写）	43 个，全部只读
固件体积 / 全局变量	501,508 B / 154,208 B	462,076 B / 133,896 B
String 跨线程	不适用（单线程）	编译期禁止；快照只带 char[]
批 0 不包含	阀/泵/位置反馈、PLC 状态字、吸附/脱附计时、水量核算、AI 推理、热泵 BACnet、控制回调（start/stop/reset/purge、时间设定、热泵设定）、Influx 直推、USB 日志。这些属性在看板上停在 12:00 之前的最后值。	

批次路线图（设计文档第 7 节）

批	内容	说明
0	骨架：28 传感器 + 15 诊断	已上板 （a43c01d）。证明两个隔离承诺；过夜考核基线 n=36。
1	阀/泵状态、PLC 状态字、16 个 onXxxChange 回调经命令队列写 PLC	启用 CtrlState.controlEnabled、drainCommands() 真正写 PLC、OTA 门恢复 !controlEnabled。
2	水量核算、自动吸附	flowRate/flowBatch 经 CtrlState 下发。
3	Influx 直推	主线程，WD_AT_PUSH。
4	AI 推理	消费快照，主线程或第三线程视 CPU 而定。
5	热泵	接口改 Modbus RTU 后再定（BACnet MS/TP 那一路不再投入）。
6	USB 日志	WD_AT_USB；注意 shared.h 多翻译单元问题。

每个批次的原则相同：新功能要么落在 PLC 线程（只碰 eip、只写快照/读 CtrlState），要么落在主线程（只碰 Cloud*、只读快照/写 CtrlState 和命令队列）。**如果一个功能非得同时碰 eip 和 Cloud*，那它必须被拆成两半，中间经过 shared.h。**

15. 批 1 之前要知道的事

全分支终审（2026-09-04）认定两个隔离承诺在代码上成立、无需在过夜前修改；下面是它留给批 1 的清单，完整版在仓库 `docs/batch0-final-review.md`。

1. `panelLeds()` 在 `#if ENABLE_SERIAL_DEBUG` 里——生产构建关掉串口后面板灯会全灭。移出来。
2. `shared.h` 的全局对象是头文件里的 `static`：单翻译单元正确，第二个 `.cpp` 包含它会静默得到第二把锁。加一个非 `static` 守卫符号让它变成链接错误。
3. PLC 线程一拍只在开头喂一次狗；批 1 加阀和状态字读取后一拍可能逼近 60 s。在每组 MSP、每个 DINT 读之后补 `wdBeatPlc()`。
4. 云探针 DNS 失败不缓存，互联网断开时每 5 s 可能阻塞 15–30 s。缓存否定结果。
5. `plcStalled` 在 PLC 线程尚未发布过任何快照时 (`stampMs=0`) 会误判；加 `seq != 0` 前置条件。
6. 重连分支不清 `ok[]`，它保留上一拍的真值；批 2 水量核算不能把它当“本拍读到”。
7. 归属断言只在 `pollSensorsInto` 一处；放进重连分支和 `drainCommands()`，或直接放进 EtherNet/IP 的 `sendRRData()`。
8. LOG 两个线程都用：没接控制台时 `USBSerial::write` 立刻返回，零风险；接着控制台时是交叉乱码不是死锁；控制台停止排空时两线程可能一起卡住 60 s 后复位。给 LOG 加锁或让 PLC 线程不写串口。
9. `loopStallMs` 只增不减。改滑动窗口。
10. 没有编译期检查保证 28 个 `TAKE` 恰好覆盖 0–27 各一次。加 `constexpr` 位掩码断言。
11. `tools/build.sh` 没有告警门。
12. `cloud_side.h` 头注释“seven String properties”应改为一个。

批 1 上线前的一个陷阱。 `ArduinoloTCloud` 会在首次同步时把云端存着的可写属性值当 `onChange` 交下来。批 0 期间看板上的按钮仍能按、云端仍会存值、板子不消费；批 1 一连上云，那些存着的值（比如一次误按的 `start`）会瞬间发到 PLC。批 1 之前先把云端这些值归位。

回滚路径。 旧固件没有归档二进制。回滚 = 在 `opta-plc-gateway-ip2` 里跑它的 `tools/build.sh` 再 OTA，或到板子跟前 DFU。

16. 术语表

术语	解释
EtherNet/IP、CIP	Allen-Bradley PLC 的以太网工业协议；CIP 是其上的通用工业协议，本固件用显式报文按标签名读写。端口 44818。
MSP	Multiple Service Packet，一条 CIP 报文里打包多个读请求。本固件一组最多 12 个标签。
REAL / DINT	CIP 数据类型：32 位浮点 / 32 位有符号整数。IP2 上 6 个点以 DINT 原始计数给出，固件乘缩放加偏移。
快照 PlcSnapshot	PLC 线程一拍的全部结果，纯数据结构，约 220 字节，加锁整体拷贝。
seq	快照序号。主线程凭它判断"有没有新的一拍"。
拍 tick	PLC 线程的一个周期，2000 ms。
心跳 beat	线程告诉喂狗线程"我还活着"的时间戳。
stall	某线程自上次心跳以来的时间；喂狗线程取两线程中的较大值。
WD_AT_*	位置码 0-13，线程在进入可能阻塞的段之前登记。
探针 probe	带超时的原生 TCP 开关一次，只看端口有没有人听，不传数据。
Mail	mbed RTOS 的定长消息队列（内存池 + 队列）。命令队列用它，16 槽。
trivially copyable	C++ 类型特征：可以 memcpy 而不破坏语义（无 String、无指针、无虚函数）。快照的编译期门槛。
ON_CHANGE / PUB_DELTA	ArduinolotCloud 的属性更新策略：值变了才发 / 变化超过阈值才发。
n=	KVStore 里的启动计数，出现在 bootReason 字符串里。不变 = 没重启。
OTA	经 Arduino Cloud 无线刷固件。回调返回 true 才允许应用。
StackWatch	手工栈水位：刷图案、数没被覆盖的字。每线程一个实例。
KVStore	Arduino 的键值持久存储（QSPI flash）。存 WiFi 凭据、启动计数。

相 关 文 档 ： 设 计 [docs/superpowers/specs/2026-09-04-opta-gateway-mt-design.md](#)； 批 0 计 划 [docs/superpowers/plans/2026-09-04-opta-gateway-mt-batch0.md](#)； 终 审 [opta-gateway-mt/docs/batch0-final-review.md](#)； 仓库 [opta-gateway-mt/README.md](#)。